A Comparative Study of Monolithic and Modular Architectures in a Dataset Driven Case Tracking Web Application

Mireya Francesca Vella
Institute of Information and Communication Technology
Malta College of Arts, Science and Technology
Paola, Malta
mireya.vella.i85114@mcast.edu.mt

***Abstract*—This study compares monolithic and modular backend architectures using two functionally equivalent FastAPI implementations of a case-tracking application. Maintainability was assessed through lines of code distribution and cyclomatic complexity analysis, testability through a shared 29-test API contract suite and mock injection precision, and change impact through two controlled modification scenarios. The modular architecture achieved a 42 per cent reduction in average cyclomatic complexity and enabled layer-boundary-accurate test isolation, whilst both implementations passed all 29 tests. The findings support the hypothesis that a modular architecture is more maintainable and testable than a monolithic architecture.**

***Index Terms*—software architecture, monolithic, modular, maintainability, testability, cyclomatic complexity**

I. Introduction

A. Description of Theme and Topic Rationale

Backend architecture refers to how a server-side system is organised into components, responsibilities, and dependency boundaries. A common architectural choice in backend development is whether to build a monolithic system, in which most functionality is implemented within one tightly connected codebase, or a modular or layered system, in which responsibilities are separated into clearer units such as routing, services, and data access [5]. This choice is important because architecture affects how easily a system can be changed, tested, and maintained over time. Research on architectural smells, co-change behaviour, and modularity shows that unclear boundaries and poor structure can increase change propagation and reduce maintainability, which supports the need for evidence-based evaluation of architectural decisions [1], [3], [4].

This study focuses on a dataset-driven case-tracking application in order to compare these approaches in a controlled manner. Two backends are built with the same user-facing behaviour, including importing a synthetic JSON dataset into PostgreSQL, validating and normalising records, logging import runs, and exposing search and trend functionality. Comparing two implementations with identical requirements is appropriate because it allows differences in maintainability, testability, and operational performance to be measured under consistent conditions rather than assumed. This type of comparative evaluation is also consistent with prior work on software re-modularisation and architecture-level assessment [4], [7].

B. Positioning and Research Onion

This study adopts a pragmatist philosophy because it aims to develop and evaluate a working software artefact and to draw conclusions from practical, measurable evidence. This position is appropriate because the outcome is assessed using objective metrics collected from the implemented systems rather than the researcher's personal opinion. The study follows a deductive approach, as it seeks to test the predefined hypothesis that a modular or layered architecture provides better maintainability and testability than a monolithic design.

The research strategy is a controlled software experiment based on comparative benchmarking. Two backend implementations are developed with identical requirements, identical dataset ingestion behaviour, and the same API contract. The analysis uses a mono-method quantitative strategy, since the research questions are addressed through numerical measures that can be compared across both architectures, including structural metrics, testing results, and performance timings. The time horizon is cross-sectional with repeated runs, meaning that both systems are evaluated at the same stage of completion while measurements are repeated to reduce random variation in runtime results. Recent research also shows growing interest in modular monolith design and stronger modular boundary practices, while emphasising the value of architecture evaluation through modularity metrics and re-modularisation analysis [4], [5], [7].

C. Background to this Research Theme

Architectural quality is closely related to maintainability, and architectural smells are recurring patterns that often indicate deeper design problems. Systematic evidence shows increasing interest in detecting such smells as part of maintaining long-term system quality [1]. Studies further show that co-changes and modularity are useful for understanding how architecture affects system evolution and maintainability [3], [4].

D. Hypothesis, Research Aim, and Purpose Statement

This study tests the hypothesis that a modular or layered architecture is more maintainable and testable than a monolithic architecture.

The purpose of this controlled comparative experiment was to test the theory that architectural modularity improves software quality by relating backend architecture type to maintainability and testability outcomes, while controlling for identical requirements, dataset, database schema, API contract, test suite, and runtime environment across both implementations. The independent variable was backend architecture type, defined as a monolithic backend or a modular or layered backend. The dependent variables were maintainability and testability, defined using tool-based structural measures and automated test evidence.

II. Literature Review

This section reviewed how recent research approached software architecture quality evaluation, with particular focus on architectural smells, maintainability, modularity, and remodularisation. The review used the research onion framework to structure findings, examining research philosophy, methodological approach, strategy, time horizon, and evaluation procedures across studies that tested architecture-related hypotheses [1], [5].

Across the reviewed studies, a clear positivist orientation was evident, with researchers prioritising objective, measurable outcomes extracted from software artefacts like source code, repository history, and static analysis results [1], [3], [6]. Most studies followed a deductive approach, testing specific expectations such as whether architectural smells correlated with change behaviour, whether modularity could be measured more accurately, or whether restructuring improved quality [3], [4], [6], [7]. Common strategies included empirical mining studies and tool-supported analysis, alongside evidence synthesis via mapping studies and workshop reviews [1], [5]. Time horizons varied: many took a cross-sectional snapshot of systems, whilst others examined version history to capture changes over time [3].

All sources reviewed were academic publications, including peer-reviewed journal articles, conference papers, and workshop papers [1]–[7]. Whilst vendor documentation, architecture blogs, and practitioner posts can inform implementation decisions, they were not included as primary evidence here because they typically lacked controlled variables, reproducible methods, or peer review [1], [5].

A. Methodologies of Similar Studies

Mumtaz et al. [1] conducted a systematic mapping study on architectural smell detection. The methodology applied structured search and selection procedures, then categorised studies by detection approaches, data sources, tool support, and validation methods. This was strong for breadth and transparency, and it supported positioning by showing that detection practices were diverse and not fully standardised. However, mapping studies did not directly test outcomes such as improved maintainability, so they contributed classification and gap identification rather than causal evidence.

Sas et al. [3] explored the relation between co-changes and architectural smells through repository mining. Co-change behaviour was used as a proxy for coupling and change propagation, then analysed alongside smell presence. The strength was that it was grounded in real maintenance activity. The limitation was that commit granularity and workflow conventions could distort co-change patterns, which could threaten internal validity if not carefully controlled.

Pisch et al. [4] proposed M-score as an empirically derived modularity metric. The methodology focused on improving measurement quality, which was important because modularity was often used as a dependent variable in architecture evaluation. Stronger metrics supported clearer comparison across systems. A limitation was that modularity is multidimensional, so M-score should be interpreted alongside complementary indicators such as complexity, test outcomes, and change impact.

Esposito et al. [6] investigated correlations between architectural smells and static analysis warnings. Methodologically, this triangulated architecture-level detection with another measurement source (static analysis), which could improve construct validity by showing alignment between different quality signals. However, correlations required careful interpretation because static warnings varied in severity and did not uniformly reflect architectural structure.

Schröder et al. [7] presented a search-based remodularisation case study at Adyen. Case study methodology supported practical relevance by evaluating restructuring in a realistic industrial setting. This provided strong context and feasibility evidence, but generalisability was reduced because results could depend on the organisation, codebase constraints, and local engineering practices.

Tan et al. [2] proposed an effort estimation approach for clustering-based remodularisation in a conference companion venue. It supported framing restructuring cost and feasibility, but it was lighter empirical evidence compared to full-length journal studies.

Similarly, Su and Li [5] discussed modular monolith as a trend in a workshop context, supporting conceptual positioning rather than providing strong controlled evaluation.

B. Comparisons of Studies

Three methodological patterns emerged across the reviewed research, as summarised in Table **??**. Synthesis work such as [1] delivered structured overviews and identified gaps, but did not directly test architecture outcomes in controlled settings. Repository-based empirical studies like [3] and [6] scaled well and used automation, yet often produced association evidence because confounding variables were difficult to control. Metric-based and case study research such as [4] and [7] provided detailed, practical evaluation, though generalising findings across different systems and organisations was challenging.

These differences justify the controlled comparative benchmarking approach adopted here. Table **??** further details the research design, data collection methods, and evaluation tools employed across the reviewed studies. By fixing requirements, dataset rules, schema, API behaviour, and test procedures, stronger internal validity can be achieved compared to observational mining studies, whilst still producing evidence relevant to actual backend development practice.

III. Research Methodologies

This research evaluated whether a modular or layered backend architecture improved maintainability and testability relative to a monolithic backend when both implementations delivered identical functionality under controlled conditions. The study was applied research as it produced two working software artefacts and evaluated them using objective evidence derived from automated tests, tooling, and repeated benchmark measurements. The methodological choices aligned with established practices in architecture quality research, which typically depends on tool-supported measurement and empirical evidence to investigate maintainability, modularity, and change impact [1], [3], [4], [6], [7].

Based on this aim, the following research questions were formed:

RQ1: To what extent does a modular or layered backend architecture improve maintainability compared to a monolithic backend when implementing the same requirements?
RQ2: To what extent does a modular or layered backend architecture improve testability compared to a monolithic backend under the same test suite and contract?
RQ3: How does architecture type affect operational performance for dataset ingestion and API responsiveness under the same environment and workload?
RQ4: How does architecture type affect change impact when implementing a controlled change request on the same codebase features?

A. Research Design

A pragmatist philosophy was adopted because this research aimed to draw practical conclusions from measurable evidence obtained from implemented systems, rather than relying on theoretical assumptions alone. A deductive approach was used to test the predefined hypothesis that a modular or layered backend would exhibit superior maintainability and testability compared to a monolithic backend. This hypothesis was grounded in existing research linking architecture structure, architectural smells, modularity, and change coupling to maintainability outcomes [1], [3], [4], [6], [7].

The research design was classified as a **quasi-experimental comparative design** employing **quantitative benchmarking**. This approach consisted of a controlled software experiment where two implementations were evaluated side-by-side under identical conditions to isolate the effects of architectural choice from confounding variables. Rather than random assignment, systematic measurement was employed, an approach well-suited to architecture evaluation in software engineering contexts. The research integrated three complementary testing methodologies: (1) **performance testing**, measuring ingestion throughput and API response latency, (2) **code quality analysis**, using modularity metrics and structural complexity measurements, and (3) **empirical comparative analysis**, examining test coverage, test quantity, and change impact [4], [6], [7].

A mono-method quantitative design was employed because the research questions were answered exclusively through comparable numerical measures. The time horizon was cross-sectional with repeated measurements, as both systems were evaluated at equivalent stages of development, with measurements replicated across multiple runs to reduce the influence of random runtime variation.

B. Environment and Technologies Used

Both backend implementations used FastAPI (Python) with PostgreSQL. The system included a dataset ingestion pipeline that validated, normalised, deduplicated, and inserted records, whilst logging import run summaries. Fairness was maintained through the same database schema and migrations, the same dataset format and rules, and identical API behaviour validated through shared integration tests.

The dataset was synthetically generated using a dedicated Python script (`generate_data.py`). The script produced 5,000 JSON records, each representing a case with seven fields: a unique case number (prefixed `RD2` followed by a six-digit identifier), service centre, case type (one of five categories), case status (one of nine possible values), a status-derived title, application filed date, and last updated date. Case numbers were guaranteed unique through deduplication at generation time. Filed and updated dates were drawn randomly from the range January 2023 to August 2025, with a business rule enforced such that cases in a *received* status had an updated date equal to their filed date. The random seed was fixed at 42 to ensure reproducibility. The resulting `cases.json` file was used as the sole input for both backend implementations, ensuring that neither architecture had any advantage in data quality or volume.

Evaluation evidence was collected using: (1) tool-based structural measurement via the Radon static analysis tool for cyclomatic complexity, including modularity indicators such as M-score where applicable [4]; (2) architecture quality indicators informed by smell detection research [1], [3], [6]; and (3) test-suite execution time as a proxy for framework-level performance overhead under mocked conditions, as live throughput and latency benchmarks were not conducted. Structural measurements were automated to eliminate subjective interpretation.

C. Experiment Design

Two artefacts were implemented: (1) a monolithic backend where routing, business rules, and data access were tightly integrated, and (2) a modular backend where routing, business

rules, and data access were separated into distinct modules with clearly defined boundaries. This two-implementation structure was inspired by the controlled evaluation approach used in [7], where architectural restructuring was assessed by comparing a before-and-after state under consistent conditions, and by [4], which evaluated modularity by applying a consistent measurement framework across multiple systems. Both versions were evaluated under identical constraints: the same requirements and outputs, identical database schema and migrations, identical dataset ingestion rules, and an identical automated test suite. Fixing these constraints follows the internal validity principle applied in [3] and [6], where confounding variables were minimised by holding non-architectural factors constant across compared systems.

The evaluation framework captured three measurement categories corresponding to the research questions: performance metrics (ingestion throughput in records per second, API response latency in milliseconds), structural quality metrics (modularity scores, cyclomatic complexity, lines of code per module), and change propagation analysis. A controlled change request was applied to both versions to measure how changes propagated through the codebase, specifically files modified and lines changed, drawing on established co-change rationale from prior evolution studies [3], [7]. All measurements were automated to eliminate subjective interpretation. By holding all other factors constant, differences in outcomes could be attributed to architectural choices rather than implementation style or environment variation.

D. Validity, Reliability, and Generalisability

Internal validity was strengthened through standardised controls across both implementations. This approach minimised the risk that observed differences stemmed from feature drift or environmental variation, a common validity threat in observational architecture studies [3], [6]. Construct validity was supported by triangulating multiple indicators of maintainability and testability rather than depending on a single metric, which was particularly important given that modularity is inherently multi-dimensional [4]. Reliability was ensured through complete automation of measurements and replicated runs, consistent with practices in empirical software engineering where automated tooling and repeated trials are used to reduce measurement error and random variation [4], [7]. Whilst generalisability was limited to the specific application and technology stack employed, the clearly documented procedures enabled other researchers to adapt or replicate the approach, consistent with how industrial case study evidence is typically contextualised in software engineering research [7].

E. Ethical Considerations

This research did not involve human participants. The dataset employed was synthetically generated, thus eliminating any risk of mishandling personal or sensitive information. The study reported methodological choices, controls, and limitations with transparency, and explicitly acknowledged tool limitations to avoid overstating the scope of conclusions [1], [4].

IV. Findings & Discussion of Results

RQ1: Maintainability

Maintainability was evaluated through LOC distribution across source files and cyclomatic complexity computed using the Radon static analysis tool [4].

Table **??** presents the LOC breakdown. The monolithic implementation placed all routing, validation, business logic, and data access into a single 632-line file. The modular implementation distributed equivalent functionality across six dedicated files totalling 857 lines. The 35.6% increase reflects the structural overhead of explicit layer boundaries and typed interface definitions, a trade-off consistent with Su and Li [5].

Table **??** summarises cyclomatic complexity. The monolithic `import_json` reached CC = 14 (grade C), combining validation, deduplication, database upsert, and import logging in one function. The modular version split these responsibilities across three dedicated functions, the most complex of which reached only CC = 9 (grade B), yielding a modular average of 2.24, a 42% reduction from 3.87, with no function exceeding grade B. This is consistent with Sas et al. [3] and with Pisch et al. [4], whose M-score framework treats complexity distribution as a key modularity indicator.

Both suites produced 13 deprecation warnings for `datetime.utcnow()`; in the monolithic suite the warning pointed to line 379 of `main.py` inside a dense multi-purpose function, whilst the modular warning pointed to `services/cases.py:56`, where the filename alone identifies the responsible layer. This difference in fault discoverability is consistent with Mumtaz et al. [1] and Esposito et al. [6].

RQ2: Testability

Testability was evaluated by executing the same 29-test API contract suite against both implementations and comparing mock injection precision; Table **??** presents the execution results.

Both implementations passed all 29 tests, confirming functional equivalence as a prerequisite for valid comparison.

The more informative distinction is mock injection precision (Table **??**): in the modular version each mock target was patched at its specific layer boundary, enabling fault attribution to a precise architectural layer, a property the monolithic design cannot offer without refactoring [7]. Esposito et al. [6] found that coupling correlated with observable quality outcomes; layer separation makes each concern independently testable.

RQ3: Operational Performance

Performance was evaluated using test-suite collection time as a proxy for framework overhead under mocked conditions. The 0.14 s difference (36%) is attributable to Python resolving six additional module imports and not to request processing logic. Crucially, both architectures share an identical

FastAPI routing layer, the same PostgreSQL schema, and no performance-relevant differences in query structure, meaning the modular file split imposed no measurable framework-level performance penalty. Live throughput and latency benchmarks were not conducted as all database calls were replaced by in-memory mock objects; a live benchmark using a load-testing harness against the run protocol in Section III would complete the RQ3 evaluation.

RQ4: Change Impact

Change impact was assessed through two controlled scenarios: (1) fixing the deprecated `datetime.utcnow()` call; (2) adding a `case_type` filter to the search endpoint across route, service, and SQL layers. Tables **??** and **??** summarise the results.

In Scenario 1 both architectures touched one file, but locating the change in 632 lines of monolithic logic carried higher side-effect risk than navigating directly to a single-responsibility service file. In Scenario 2 the modular architecture spread changes across three bounded layer files, the only metric where it had a higher raw count, yet with a lower regression risk overall. Both patterns are consistent with the co-change findings of Sas et al. [3] and the Adyen remodularisation case study of Schröder et al. [7].

Across all four research questions, the modular architecture demonstrated measurable advantages in complexity distribution, fault discoverability, mock injection precision, and change traceability, outweighing the monolithic benefits of lower LOC and marginally faster test collection. These findings support the hypothesis that a modular architecture is more maintainable and testable.

In relation to prior research, these findings both extend and qualify the existing literature. Sas et al. [3] identified co-change propagation in production systems where team size, commit history, and accumulated technical debt act as confounds; this study reproduces the same directional pattern in a controlled artefact setting, strengthening the causal inference that boundary structure itself drives co-change behaviour. Esposito et al. [6] found that coupling correlated with observable quality outcomes at system scale; this study provides a more granular mechanism: mock injection layer accuracy is a directly measurable coupling proxy that can be verified from test code alone, without requiring a full quality history. Su and Li [5] frame the LOC overhead of modularisation as a trade-off requiring contextual judgment; this study quantifies it: the 35.6% LOC increase produced a 42% reduction in average cyclomatic complexity and lifted grade-A functions from 80% to 97.3%, providing evidence that the overhead is justified in this context. No reviewed study provided a protocol for benchmarking performance overhead attributable specifically to modular file structure, a gap future work should address alongside extending evaluation to additional technology stacks and applying the effort estimation framework of Tan et al. [2] to quantify remodularisation cost. Compared with the reviewed literature, which predominantly analysed existing production systems, the use of two controlled, functionally equivalent artefacts represents a distinct methodological contribution.

V. Conclusion

This research tested the hypothesis that a modular backend architecture is more maintainable and testable than a monolithic backend, using two purpose-built, functionally equivalent FastAPI artefacts evaluated through structural analysis, automated testing, and controlled change scenarios.

The findings confirm that the modular architecture is more maintainable and testable. RQ1 revealed a 42% reduction in average cyclomatic complexity and improved fault discoverability under the modular design. RQ2 demonstrated layer-boundary-accurate mock injection, enabling fault attribution to a specific architectural layer in a way the monolithic design cannot support without refactoring. RQ3 was partially addressed through proxy data; no structural performance differences were identified across the shared framework configurations. RQ4 showed that the modular design constrained co-change scope to well-defined layer boundaries and reduced regression risk. These conclusions affirm the hypothesis posed at the outset of this study.

Whilst the methodology yielded clear and consistent findings, some limitations should be acknowledged. First, the study was constrained to a single application domain and a single technology stack (FastAPI with PostgreSQL). This means that the observed advantages of the modular architecture in terms of complexity and change traceability may not transfer uniformly to systems with different structural characteristics, such as event-driven architectures or microservice deployments, where the definition of a module boundary differs fundamentally. Second, the performance benchmark for RQ3 was limited to test-suite collection time, as all database calls were replaced by in-memory mock objects for test isolation. This means that no evidence was produced regarding ingestion throughput or per-endpoint latency under realistic load, and therefore no conclusions about production-level operational performance can be drawn from the RQ3 findings. Third, whilst the synthetic dataset ensured reproducibility and eliminated data privacy concerns, it was generated under fixed statistical parameters and may not capture the irregular patterns, missing values, or schema drift that characterise real-world data ingestion pipelines; the robustness of the import validation logic under such conditions therefore remains untested.

To address such limitations and further advance research in this area, several avenues for further investigation are proposed. Replicating the comparison with a live database and load-testing harness would complete the RQ3 evaluation as described in Section III. Extending the study to additional stacks such as Node.js or Spring Boot would test whether the maintainability advantages generalise beyond the Python FastAPI context. Applying the effort estimation framework of Tan et al. [2] to quantify restructuring cost would enable practitioners to weigh the benefits of modularisation against its reorganisation overhead.

REFERENCES

[1] H. Mumtaz, P. Singh, and K. Blincoe, "A Systematic Mapping Study on Architectural Smells Detection," Journal of Systems and Software, vol. 173, Art. no. 110885, 2021, doi: 10.1016/j.jss.2020.110885.

[2] A. J. J. Tan, C. Y. Chong, and A. Aleti, "Closing the Loop for Software Remodularisation: REARRANGE, An Effort Estimation Approach for Software Clustering Based Remodularisation," in Companion Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE-Companion), 2023, pp. 326–327, doi: 10.1109/ICSE-Companion58688.2023.00090.

[3] D. Sas, P. Avgeriou, R. Kruizinga, and R. Scheedler, "Exploring the Relation Between Co-changes and Architectural Smells," SN Computer Science, 2021, doi: 10.1007/s42979-020-00407-5.

[4] E. Pisch, Y. Cai, R. Kazman, J. Lefever, and H. Fang, "M-score: An Empirically Derived Software Modularity Metric," in Proc. ACM/IEEE Int. Symp. on Empirical Software Engineering and Measurement (ESEM), pp. 382–392, 2024, doi: 10.1145/3674805.3686697.

[5] R. Su and X. Li, "Modular Monolith: Is This the Trend in Software Architecture?," in Proc. 1st Int. Workshop on New Trends in Software Architecture (SATrends '24), 2024, doi: 10.1145/3643657.3643911.

[6] M. Esposito, M. Robredo, F. Arcelli Fontana, and V. Lenarduzzi, "On the Correlation Between Architectural Smells and Static Analysis Warnings," Software Quality Journal, 2025, doi: 10.1007/s11219-025-09730-7.

[7] C. Schröder, A. van der Feltz, A. Panichella, and M. Aniche, "Search Based Software Re Modularization: A Case Study at Adyen," in Proc. IEEE/ACM 43rd Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2021, pp. 81–90, doi: 10.1109/ICSE-SEIP52600.2021.00017.